

问题四

冯裕凯

2026-09-19

一、任务与基线

题目要求探索多隐层神经元的不同参数设置对训练过程和结果的影响，至少覆盖三类参数、每类 2 3 种设置。本实验直接基于上学期机器学习自己编写的 homework2/code 里已有的 Fashion-MNIST 卷积网络 (model.py / train.py / data.py)，网络结构本身没有改动，只把超参变成可以传入的参数。

网络结构 (Fashion_Mnist_Model):

层	配置	输出尺寸	参数量
Conv2d(1, C_1 , 3)	SiLU	$26 \times 26 \times C_1$	$C_1 \times (1 \times 3 \times 3 + 1)$
AdaptiveAvgPool2d(16)	-	$16 \times 16 \times C_1$	0
Conv2d(C_1 , C_2 , 4)	SiLU	$13 \times 13 \times C_2$	$C_2 \times (C_1 \times 4 \times 4 + 1)$
AdaptiveAvgPool2d(8)	-	$8 \times 8 \times C_2$	0
Linear($8 \cdot 8 \cdot C_2$, 10)	-	10	$10 \times (64C_2 + 1)$

其中 C_1, C_2 就是本次被扫描的“隐层神经元个数”，原代码写死为 (8, 16)。

二、对基线代码的修改

只做了加法式的修改，默认参数下行为与原代码完全一致：

- model.py:** Fashion_Mnist_Model.__init__ 增加 width: tuple[int,int] = (8,16) 参数，两个卷积层的通道数由 c1, c2 = width 给出。默认值 (8,16) 与原来的硬编码值相同，参数量和前向过程完全不变。
- train.py:** train() 增加 width / optimizer / weight_decay / momentum / seed / data_root / verbose / save 参数，并返回每个 epoch 的 loss、train_acc、test_acc 列表；evaluate() 增加 verbose 开关。训练/评估的循环逻辑没有改动。
- 新增文件：homework3/code/experiment.py (实验 notebook 的源文件)、ipynb2typst.py (把 notebook 渲染成 pdf)、merge_pdf.py (合并报告与代码)。

三、实验设计

固定随机种子 seed=0，统一训练 15 个 epoch，batch size 256，输入用原来的 ToTensor() (不做数据增强)，用 Adam + 交叉熵。每组内只改一个参数，其余取 baseline 值 (width=(8,16), lr=1e-3, Adam, weight_decay=0)。

组	被扫描的参数	设置 1	设置 2 / 设置 3
A	网络架构：卷积层通道数 (C_1, C_2)	(4, 8) 更窄	(8, 16) baseline / (16, 32) 更宽
B	学习率 lr	10^{-4}	10^{-3} baseline / 10^{-2}
C	优化器与 weight decay	SGD(m=0.9, lr= 10^{-2})	Adam baseline / AdamW(wd= 10^{-2})

baseline 在三组里是同一个配置，用配置名去重后只训练一次，三组共用同一份曲线，所以 A/B/C 三组一共只需要 7 次训练。每次训练都在线记录训练损失和测试精度，用来观察“训练过程”而不只是最终结果。最后另外用 $\text{seed} = 1, 2$ 各重跑一次 baseline，用来估计单次运行的随机波动（组 C 的差异太小，不给个误差尺度就没法判断）。

四、结果

九行结果（7 次训练，baseline 在三组里各出现一次）：

组	配置	参数量	最终测试精度	最佳测试精度	最终训练损失
A	(4, 8) 更窄	5 714	88.75%	88.75%	0.2783
A	(8, 16) baseline	12 442	89.99%	89.99%	0.2345
A	(16, 32) 更宽	28 970	90.92%	90.92%	0.1898
B	$\text{lr} = 10^{-4}$	12 442	86.88%	86.88%	0.3417
B	$\text{lr} = 10^{-3}$ (baseline)	12 442	89.99%	89.99%	0.2345
B	$\text{lr} = 10^{-2}$	12 442	89.12%	90.34%	0.2134
C	SGD ($m = 0.9, \text{lr} = 10^{-2}$)	12 442	89.93%	89.93%	0.2452
C	Adam (baseline)	12 442	89.99%	89.99%	0.2345
C	AdamW ($\lambda = 10^{-2}$)	12 442	90.06%	90.06%	0.2356

（每列的最好值加粗。）单次训练耗时在 37 48 秒之间，同一配置重跑也会差十秒左右，主要是机器负载和 CUDA 首次预热造成的，和模型大小没有明显关系。

4.1 组 A：网络架构（隐层宽度）

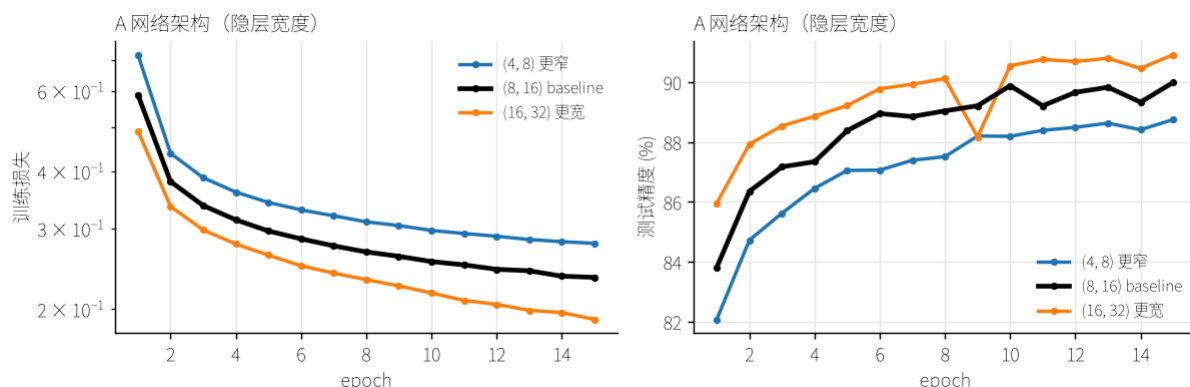


Figure 1: 组 A：卷积层通道数 (C_1, C_2) 取 (4, 8) / (8, 16) / (16, 32)。左为训练损失（对数轴），右为测试精度；黑线为 baseline。

三条曲线从第 2 个 epoch 起就分开，之后几乎没有交叉：宽度越大，训练损失越低、测试精度越高。但收益在迅速递减：参数量从 5 714 增到 12 442 ($\times 2.18$) 带来 1.24 个百分点，再从 12 442 增到 28 970 ($\times 2.33$) 只带来 0.93 个百分点——参数量总共翻了 5 倍多，只换来 2.17 个百分点。

还要注意 (4, 8) 那条曲线到第 15 个 epoch 仍在上升、训练损失也仍在下降，说明它主要是欠拟合，而不是“容量变大就会过拟合”。换句话说，在 15 个 epoch 的预算下，加宽的收益还没被过拟合抵消掉。

4.2 组 B：学习率

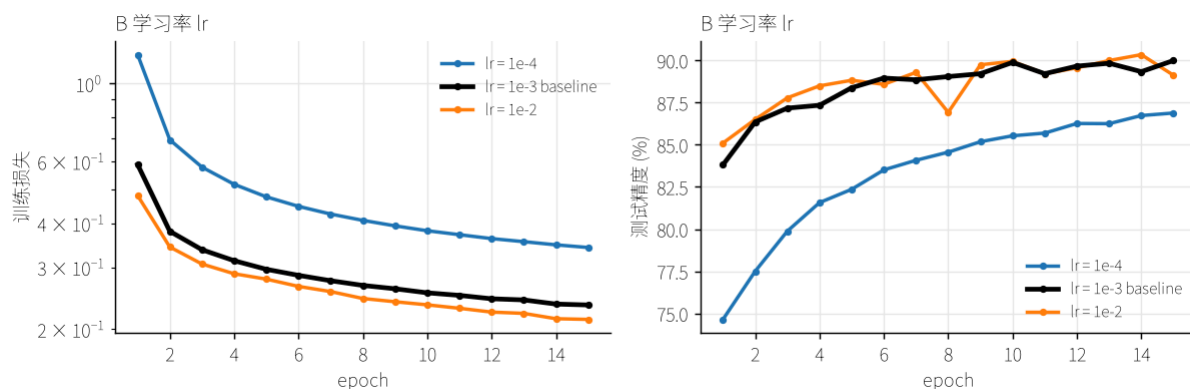


Figure 2: 组 B: lr 取 10^{-4} / 10^{-3} / 10^{-2} , 其余同 baseline。

这是三组里差异最大的一组：

1. $lr = 10^{-4}$ ：训练损失从 1.1 缓慢降到 0.34，到第 15 个 epoch 测试精度才 86.88% 且仍在爬升。它和 baseline 之间 3.11 个百分点的差距几乎全是“没走完”造成的，属于欠拟合。
2. $lr = 10^{-3}$ (baseline)：下降平稳，测试精度单调爬升到 89.99%。
3. $lr = 10^{-2}$ ：前期最快，第 5 个 epoch 就达到约 89.3%；随后开始不稳定，第 8 个 epoch 掉到 86.7% 附近，之后回到 90% 上下震荡，最后停在 89.12%。它的最佳值 90.34% 其实高于 baseline 的 89.99%，只是保持不住。

4.3 组 C：优化器与 weight decay

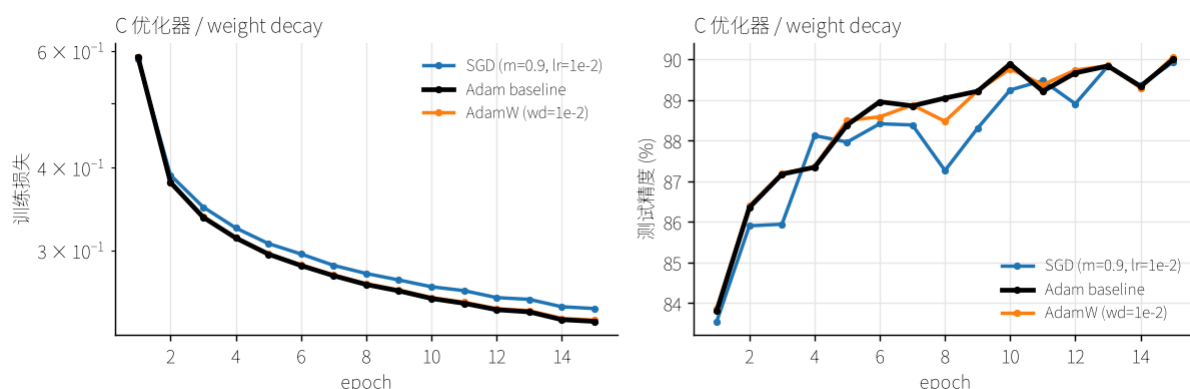


Figure 3: 组 C: $SGD(m = 0.9, lr = 10^{-2})$ / Adam / AdamW($\lambda = 10^{-2}$)。三条曲线几乎重合。

这一组三条曲线在图上基本重叠：最终测试精度 89.93% / 89.99% / 90.06%，极差 0.13 个百分点。能看出的仅有两点趋势：

1. SGD 的训练损失略高于两个 Adam (0.2452 对 0.2345)，前几个 epoch 的测试精度也略低，说明在 15 个 epoch 内 SGD 还没完全追上——它需要更多 epoch 或更大的 lr 。
2. AdamW 的最终训练损失 (0.2356) 比 Adam (0.2345) 略高，测试精度却略高 (90.06% 对 89.99%)。这正是 weight decay 该有的方向：压低权重范数、牺牲一点训练集拟合、换一点泛化；但幅度只有 0.07 个百分点，单次运行说明不了问题。

4.4 参数量与精度的关系

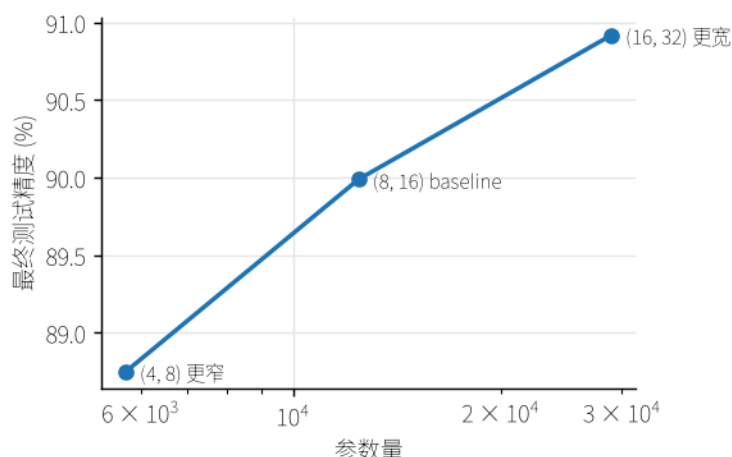


Figure 4: 组 A 的参数量（对数轴）与最终测试精度。

五、分析

5.1 三类参数的敏感度排序

把三组放在一起，结论很清楚：**学习率 > 网络宽度 > 优化器/weight decay**。

组	组内最终测试精度的极差	判断
B 学习率	3.11 个百分点 (86.88 → 89.99)	影响最大，且有质变（欠拟合 / 发散）
A 网络架构	2.17 个百分点 (88.75 → 90.92)	影响明显，但收益递减
C 优化器 / wd	0.13 个百分点 (89.93 → 90.06)	在 15 个 epoch 内无实质差别

5.2 为什么学习率的影响最大

10^{-4} 的问题是步子太小：15 个 epoch 里参数根本没走到谷底，训练损失还停在 0.34（baseline 是 0.23），所以它属于欠拟合，和模型容量无关——把它换成更宽的网络也一样救不回来。

10^{-2} 的问题相反：训练损失最终 0.2134 比 baseline 的 0.2345 还低，测试精度却更低，而且中途明显回落，说明它已经走到比较尖锐的区域、并开始拟合训练集里的噪声。这正好说明为什么实践中常用**学习率衰减**：前期要 10^{-2} 的速度，后期要 10^{-3} 的稳定。

5.3 为什么优化器之间看不出差别

Adam 与 AdamW 都是自适应方法，在这个任务上（不到 3 万参数、15 个 epoch、输入只是归一化到 $[0, 1]$ 的灰度图、且没有数据增强）优化本身不是瓶颈，所以 0.13 个百分点的差异 **还不到换一个随机种子带来的波动**（见 5.4）。要区分优化器，需要更长的训练、更大的模型，或者把 weight decay 调大—— 10^{-2} 对这个模型太小了，它只把最终训练损失抬高了 0.0011。

5.4 重复性检查

为了判断组 C 那 0.13 个百分点是不是噪声，把 baseline 用三个不同随机种子各训练一次：

seed	最终测试精度	与 seed=0 的差
0	89.99%	-
1	89.74%	-0.25 pp
2	89.68%	-0.31 pp

均值 89.80%，标准差 0.13 个百分点，极差 0.31 个百分点。组 C 三种设置之间的极差（0.13）恰好等于这里的标准差、且小于种子极差（0.31），所以“这一组看不出优化器差别”是站得住的结论，而不是“AdamW 略好于 Adam”。

还有一个更值得注意的现象：整个实验原样重跑一遍（同一份代码、同样 seed=0），各配置的最终测试精度也会变——例如 (16, 32) 从 90.99% 变成 90.92%， $lr=10^{-2}$ 从 89.28% 变成 89.12%。原因是 PyTorch 默认关闭了确定性卷积（`torch.backends.cudnn.deterministic = False`），cuDNN 会自动挑选非确定性的实现。所以在这个量级上，**0.1 0.3 个百分点的差异不能当作参数优劣的证据**，本报告的所有排名都限定在这套设置之内。

六、结论

1. **被扫描的三类参数里，只有学习率和网络宽度会实质改变结果。** 学习率要在“收敛速度”和“稳定性”之间取折中： 10^{-4} 太慢（欠拟合）、 10^{-2} 会震荡， 10^{-3} 在本设置下最好；实际训练应当配合学习率衰减，而不是固定一个小 lr 硬跑。
2. **加宽网络能涨点，但边际收益下降得很快。** 参数量翻 5 倍只换来 2.17 个百分点，说明这个任务上限制性能的主要不是容量；反过来，若要在同等精度下省算力，选较小的 (8, 16) 是划得来的（12 442 个参数拿到 89.99%，只比 (16, 32) 低 0.93 个百分点，参数量却只有它的 43%）。
3. **优化器和 weight decay 在本实验的规模下不是有效变量。** 15 个 epoch 内三者的差别小于种子波动，单次实验不能据此判断优劣；要评估它们必须延长训练、并至少重复多次取均值。
4. **实验局限：**只做了单种子（组 C 除外）、只训练 15 个 epoch、没有数据增强、也没有跨数据集验证；阈值以下的差异（ < 0.3 pp）不具备判别力。因此上面的排序只在这个具体设置下成立。

机器学习第三次作业 · 问题四

多隐层神经元不同参数设置对训练过程和结果的影响

本实验基于 homework2/code 里已有的 Fashion-MNIST 卷积网络。网络本身没有改动，只给 Fashion_Mnist_Model 增加了一个 width 参数（默认 (8,16)，与原来完全一致），并让 train() 能接受优化器 / weight decay / batch size 等参数、返回每个 epoch 的指标。

固定随机种子与除被扫描项之外的其它超参，逐组扫描三类参数：

组	被扫描参数	三种设置
A	网络架构：两个卷积层通道数（隐层神经元个数）	(4,8) / (8,16) / (16,32)
B	学习率 lr	1e-4 / 1e-3 / 1e-2
C	优化器与 weight decay	SGD(m=0.9, lr=1e-2) / Adam / AdamW(wd=1e-2)

baseline 为 width=(8,16)，lr=1e-3，Adam，bs=256，15 epoch，它在三组里是同一个配置，因此只训练一次、三组共用。

```
import os
import sys
import time
from pathlib import Path

import matplotlib
import matplotlib.pyplot as plt
import pandas as pd
import torch

try:
    from IPython.display import display
except ImportError:
    display = print

REPO = Path.cwd().resolve().parents[1]
HW2_CODE = REPO / "homework2" / "code"
DATA_ROOT = str(HW2_CODE / "tmp" / "data")
sys.path.insert(0, str(HW2_CODE))

from model import Fashion_Mnist_Model
from train import train

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("torch      :", torch.__version__)
print("device     :", DEVICE, torch.cuda.get_device_name(0) if DEVICE.type == "cuda"
      else "")

plt.rcParams.update({
    "svg.fonttype": "path",
    "font.sans-serif": ["Noto Sans CJK SC", "DejaVu Sans"],
    "font.family": "sans-serif",
    "axes.unicode_minus": False,
    "figure.dpi": 110,
})
```

```
torch      : 2.14.0+cu130
device     : cuda NVIDIA GeForce RTX 4060 Laptop GPU
```

1. 实验设置

所有实验统一用同一个随机种子 (seed=0)、15 个 epoch、batch size 256。每组内只改一个参数，其余取 baseline 值。

```
NUM_EPOCH = 15

BASE = dict(
    num_class=10,
    dropout=0.3,          # 注意：原代码里 dropout 参数没有接到任何 Dropout 层，实际不起作用
    num_epoch=NUM_EPOCH,
    batch_size=256,
    lr=1e-3,
    width=(8, 16),
    optimizer="adam",
    weight_decay=0.0,
    seed=0,
)

# 每组：(标签, 覆盖 baseline 的参数, 该配置的短名字)
GROUPS = {
    "A 网络架构 (隐层宽度)": [
        ("(4, 8) 更窄", dict(width=(4, 8)), "w4_8"),
        ("(8, 16) baseline", dict(width=(8, 16)), "base"),
        ("(16, 32) 更宽", dict(width=(16, 32)), "w16_32"),
    ],
    "B 学习率 lr": [
        ("lr = 1e-4", dict(lr=1e-4), "base_lr1e-4"),
        ("lr = 1e-3 baseline", dict(lr=1e-3), "base"),
        ("lr = 1e-2", dict(lr=1e-2), "base_lr1e-2"),
    ],
    "C 优化器 / weight decay": [
        ("SGD (m=0.9, lr=1e-2)", dict(optimizer="sgd", lr=1e-2), "sgd"),
        ("Adam baseline", dict(optimizer="adam", lr=1e-3), "base"),
        ("AdamW (wd=1e-2)", dict(optimizer="adamw", lr=1e-3, weight_decay=1e-2),
        "adamw"),
    ],
}

def n_params(cfg) -> int:
    """按配置建一个网络，数参数量"""
    model = Fashion_Mnist_Model(cfg["num_class"], cfg["dropout"], cfg["width"])
    return sum(p.numel() for p in model.parameters())

print("baseline 参数量:", n_params(BASE))
```

baseline 参数量: 12442

2. 开始训练

baseline 在三组里重复出现，用 key 去重，只训练一次。

```
histories = {} # key -> list(dict), 每个 epoch 的 loss / train_acc / test_acc
records = []   # 汇总表

for group, runs in GROUPS.items():
    for label, over, key in runs:
        if key in histories:
```

```

        print(f"[skip] {label} (与已训练的 {key} 相同)")
        continue
    cfg = {**BASE, **over}
    t0 = time.time()
    hist = train(
        art_dir=f"tmp/runs/{key}",
        data_root=DATA_ROOT,
        verbose=False,
        save=False,
        **cfg,
    )
    histories[key] = hist
    secs = time.time() - t0
    final = hist[-1]
    records.append({
        "组": group,
        "配置": label,
        "key": key,
        "参数量": n_params(cfg),
        "lr": cfg["lr"],
        "优化器": cfg["optimizer"],
        "weight_decay": cfg["weight_decay"],
        "最终测试精度": round(final["test_acc"], 2),
        "最佳测试精度": round(max(h["test_acc"] for h in hist), 2),
        "最终训练损失": round(final["loss"], 4),
        "耗时(s)": round(secs, 1),
    })
    print(f"[done] {label:26s} test acc = {final['test_acc']:.2f}% ({secs:.1f}s)")

```

```
[done] (4, 8) 更窄                test acc = 88.75% (48.3s)
```

```
[done] (8, 16) baseline          test acc = 89.99% (36.7s)
```

```
[done] (16, 32) 更宽             test acc = 90.92% (38.9s)
```

```
[done] lr = 1e-4                 test acc = 86.88% (37.1s)
[skip] lr = 1e-3 baseline (与已训练的 base 相同)
```

```
[done] lr = 1e-2                 test acc = 89.12% (43.3s)
```

```
[done] SGD (m=0.9, lr=1e-2)      test acc = 89.93% (45.6s)
[skip] Adam baseline (与已训练的 base 相同)
```

```
[done] AdamW (wd=1e-2)           test acc = 90.06% (41.1s)
```

```
df = pd.DataFrame(records)
df
```

	组	配置	key	参数量	lr \
0	A 网络架构 (隐层宽度)	(4, 8) 更窄	w4_8	5714	0.0010
1	A 网络架构 (隐层宽度)	(8, 16) baseline	base	12442	0.0010
2	A 网络架构 (隐层宽度)	(16, 32) 更宽	w16_32	28970	0.0010
3	B 学习率 lr	lr = 1e-4 base_lr	1e-4	12442	0.0001

	B 学习率 lr	lr = 1e-2	base_lr1e-2	12442	0.0100
5	C 优化器 / weight decay	SGD (m=0.9, lr=1e-2)	sgd	12442	0.0100
6	C 优化器 / weight decay	AdamW (wd=1e-2)	adamw	12442	0.0010

	优化器	weight_decay	最终测试精度	最佳测试精度	最终训练损失	耗时(s)
0	adam	0.00	88.75	88.75	0.2783	48.3
1	adam	0.00	89.99	89.99	0.2345	36.7
2	adam	0.00	90.92	90.92	0.1898	38.9
3	adam	0.00	86.88	86.88	0.3417	37.1
4	adam	0.00	89.12	90.34	0.2134	43.3
5	sgd	0.00	89.93	89.93	0.2452	45.6
6	adamw	0.01	90.06	90.06	0.2356	41.1

3. 训练曲线

每组画两张图：训练损失和测试精度随 epoch 的变化。

```
def plot_group(group: str, fname: str):
    runs = GROUPS[group]
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9.4, 3.2), dpi=130)
    for label, _, key in runs:
        h = histories[key]
        ep = [x["epoch"] for x in h]
        style = dict(linewidth=1.9, marker="o", markersize=3)
        if key == "base":
            style.update(linewidth=2.6, color="black", zorder=5)
        ax1.plot(ep, [x["loss"] for x in h], label=label, **style)
        ax2.plot(ep, [x["test_acc"] for x in h], label=label, **style)
    ax1.set_xlabel("epoch"); ax1.set_ylabel("训练损失"); ax1.set_yscale("log")
    ax2.set_xlabel("epoch"); ax2.set_ylabel("测试精度 (%)")
    ax1.set_title(group, loc="left", fontsize=11)
    ax2.set_title(group, loc="left", fontsize=11)
    for ax in (ax1, ax2):
        ax.grid(True, color="#e8e8e8", linewidth=0.7)
        ax.set_axisbelow(True)
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)
        ax.legend(fontsize=8.5, frameon=False)
    fig.tight_layout()
    fig.savefig(fname, bbox_inches="tight")
    display(fig)
    plt.close(fig)

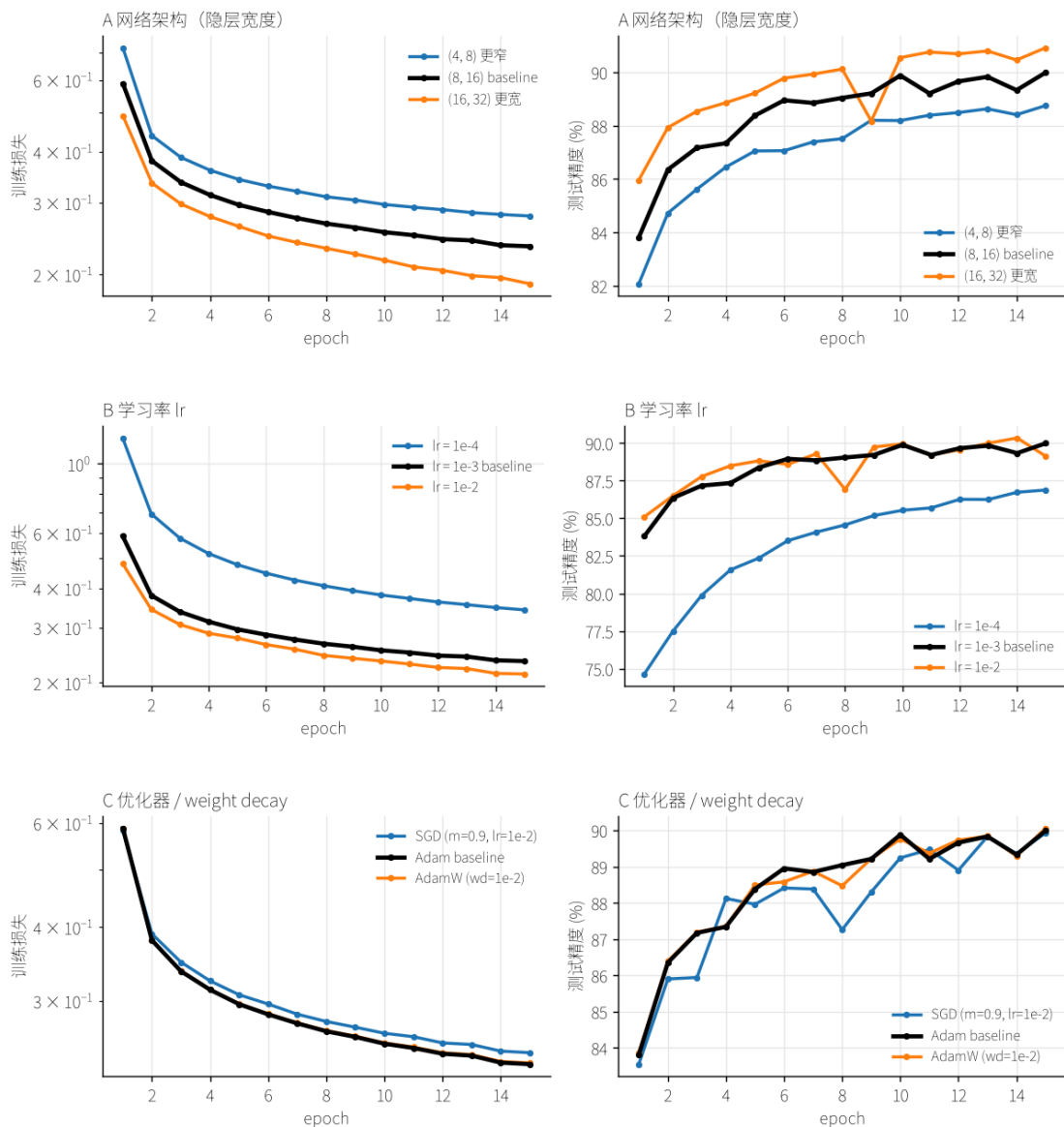
plot_group("A 网络架构 (隐层宽度)", "fig_A_architecture.png")
plot_group("B 学习率 lr", "fig_B_lr.png")
plot_group("C 优化器 / weight decay", "fig_C_optimizer.png")
```

findfont: Failed to find font weight normal for Noto Sans CJK SC, now using 100.

findfont: Failed to find font weight normal for Noto Sans CJK SC, now using 100.

findfont: Failed to find font weight normal for Noto Sans CJK SC, now using 100.

findfont: Failed to find font weight normal for Noto Sans CJK SC, now using 100.



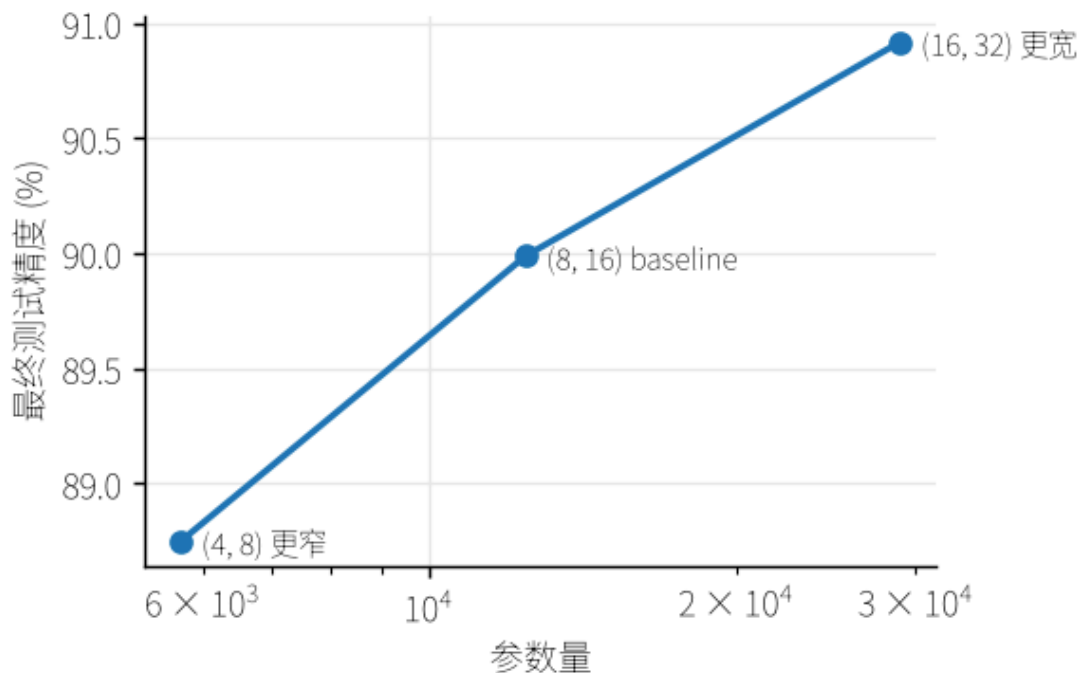
4. 汇总对比

```
summary = df.pivot_table(index="配置", values=["最终测试精度", "最佳测试精度", "参数量"],
sort=False)
summary
```

配置	最终测试精度	最佳测试精度	参数量
(4, 8) 更窄	88.75	88.75	5714.0
(8, 16) baseline	89.99	89.99	12442.0
(16, 32) 更宽	90.92	90.92	28970.0
lr = 1e-4	86.88	86.88	12442.0
lr = 1e-2	89.12	90.34	12442.0
SGD (m=0.9, lr=1e-2)	89.93	89.93	12442.0
AdamW (wd=1e-2)	90.06	90.06	12442.0

```
# 参数量与精度的关系 (组 A)
a = df[df["组"] == "A 网络架构 (隐层宽度)"]
fig, ax = plt.subplots(figsize=(4.6, 3.0), dpi=130)
```

```
ax.plot(a["参数量"], a["最终测试精度"], marker="o", color="#1f77b4", linewidth=1.9)
for _, r in a.iterrows():
    ax.annotate(r["配置"], (r["参数量"], r["最终测试精度"]),
               textcoords="offset points", xytext=(6, -4), fontsize=8.5)
ax.set_xscale("log")
ax.set_xlabel("参数量"); ax.set_ylabel("最终测试精度 (%)")
ax.grid(True, color="#e8e8e8", linewidth=0.7); ax.set_axisbelow(True)
ax.spines["top"].set_visible(False); ax.spines["right"].set_visible(False)
fig.tight_layout(); fig.savefig("fig_D_params_vs_acc.png", bbox_inches="tight")
display(fig); plt.close(fig)
```



5. 重复性检查

组 A、B 的差异有 13 个百分点，而组 C（优化器 / weight decay）三条曲线几乎重合，差不到 0.2 个百分点。要判断组 C 的差异是不是真的，得先知道单次运行的波动有多大：用不同随机种子把 baseline 重跑两遍（seed=0 复用上面已经训练的结果）。

```
seed_accs = [histories["base"][-1]["test_acc"]]
for s in (1, 2):
    h = train(art_dir=f"tmp/runs/seed{s}", data_root=DATA_ROOT, verbose=False,
             save=False,
             **{**BASE, "seed": s})
    seed_accs.append(h[-1]["test_acc"])
    print(f"seed={s} 最终测试精度 = {h[-1]['test_acc']:.2f}%")

import statistics as st
print(f"\nbaseline 三个种子的最终测试精度: " + ", ".join(f"{a:.2f}%" for a in seed_accs))
print(f"均值 {st.mean(seed_accs):.2f}% 标准差 {st.pstdev(seed_accs):.2f}% "
      f"极差 {max(seed_accs) - min(seed_accs):.2f} 个百分点")
```

```
seed=1 最终测试精度 = 89.74%
```

```
seed=2 最终测试精度 = 89.68%
```

```
baseline 三个种子的最终测试精度: 89.99%, 89.74%, 89.68%  
均值 89.80% 标准差 0.13% 极差 0.31 个百分点
```

6. 结论

详细分析见报告。